This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

Playing Multi-Action Adversarial Games: Online Evolutionary Planning versus Tree Search

Niels Justesen, Tobias Mahlmann, Sebastian Risi and Julian Togelius

We address the problem of playing *turn-based multi-action adversarial* games, which include many strategy games with extremely high branching factors as players take multiple actions each turn. This leads to the breakdown of standard tree search methods, including Monte Carlo Tree Search (MCTS), as they become unable to reach a sufficient depth in the game tree. In this paper, we introduce *Online Evolutionary Planning* (OEP) to address this challenge, which searches for combinations of actions to perform during a single turn guided by a fitness function that evaluates the quality of a particular state. We compare OEP to different MCTS variations that constrain the exploration to deal with the high branching factor in the turn-based multi-action game *Hero Academy*. While the constrained MCTS variations outperform the vanilla MCTS implementation by a large margin, OEP is able to search the space of plans more efficiently than any of the tested tree search methods as it has a relative advantage when the number of actions per turn increases.

I. Introduction

Adversarial games, in which one player's loss is the others gain, have a long tradition as testbeds in artificial intelligence (AI). In this context, playing the game well can be viewed as a search from the current state of the game to desirable future states. In fact, many well performing game-playing programs rely on search algorithms that are guided by some heuristic function that evaluates the desirability of a given state. For adversarial two-player games with a relatively low branching factors, such as Checkers and Chess, search algorithms such as Minimax together with well-designed heuristic functions perform remarkably well [1].

However, as the branching factor increases, the efficacy of Minimax search is greatly reduced. In cases where it is hard to develop or learn informative heuristic functions this problem is further compounded. A classic example is *Go*, where it took several decades of research to come up with algorithms that play at world-class human level [2], [3].

While Go has a branching factor of "only" 300 (a magnitude higher than the 30 for Chess), it is still far lower than most *turn-based multi-action adversarial games*, where each player takes multiple separate actions each turn, for example by moving multiple units or pieces. In this case, none of the mentioned methods currently perform well. Examples of multi-action turn-based adversarial games include strategy games such as *Civilization*, *Warhammer 40k*, *XCOM* or *Heroes of Might and Magic* but also card and board games such as the board game *Arimaa*. This class of problems arguably also includes many real-world problems involving coordination of multiple agents.

By including multiple actions and multiple units, the branching factor quickly reaches intractable dimensions. For example, a strategy game that allows the movement of six units every turn and each unit can perform one out of ten actions has a branching factor of a million (10^6). Standard tree search methods tend to fail with such high branching factors because the trees become very shallow. To allow tree search methods to be applied in such circumstances, some authors resort to making strong assumptions to guide how to explore actions [4]. One naive assumption, which is wrong in the general case, is independence between units, which decreases the branching factor to only 60 rather than a million.

Instead of relying on a tree-based search method, Perez et al. introduced an evolutionary algorithm-based method for playing non-adversarial games called *Rolling Horizon Evolution* (RHE; [5]). RHE evolves a sequence of game actions to perform in the near future. The agent then performs the first action in the found sequence and evolves a new sequence from scratch. This process is continued a number of times until the game is over. The RHE algorithms runs *online* during the game, differing sharply with the way evolution is typically applied in game-playing, in which a controller is evolved that determines the actions of the agent [6]–[9]. The fitness function of an action sequence is the desirability of the final state reached by performing these actions, which is estimated by some heuristic. RHE has been successful in a number of real-time environments including the Physical Traveling Salesman Problem [10] and in the General Video Game Playing benchmark [11], [12]. It is specifically designed for real-time games and thus cannot be directly applied to adversarial games. Evolution can however still be useful for these games by evolving a *sequence of actions to take during a single turn*. Finding the optimal combination of actions to perform during just one turn is a formidable search problem in itself due to combinatorial complexities.

The novel *Online Evolutionary Planning* (OEP) algorithm introduced in this paper, searches for the actions to perform in a single turn and uses an estimation of the state at the end of the turn (i.e. right before the opponent takes their turn) as a fitness function. We compare the OEP approach to Monte Carlo Tree Search (MCTS) [13], which has shown to work well for games with higher branching factors. MCTS handles higher branching factors well by building an unbalanced tree, and performs state estimations by Monte Carlo simulations until the end of the game. The advent of the MCTS algorithm caused a qualitative improvement in the performance of Go-playing programs [14], and MCTS has been part of almost every high-level Go-playing program since, including the world champion AlphaGo [2].

The domain investigated in this paper is the multi-action adversarial turn-based game called *Hero Academy*, a competitive strategy game playable on PC and iPad. Because of the

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

extremely high branching factor of Hero Academy, we also developed two variations of MCTS that attempt to limit exploration, resulting in a more focused search. The first MCTS variation has a greedy tree policy, that always selects the most valuable node, and a deterministic default policy during rollouts. The other variation prunes branches aggressively to push the search in the most promising direction.

This paper builds and expands on results previously published in conference proceedings [15]. In more detail we added: (1) two new variations of MCTS that aggressively constrain the exploration to deal with the enormous action space of multi-action games, (2) an investigation of how the methods perform with varying numbers of actions per turn, which demonstrates the scalability of OEP, (3) a user study of 111 games with users of various skill levels, showing the usefulness of applying OEP to a real game product, and (4) a deeper complexity analysis of the game Hero Academy as well as a more in-depth discussion of the results.

The new MCTS variations, Bridge Burning MCTS (BB-MCTS) and non-exploring MCTS are shown to outperform vanilla MCTS in this domain. While OEP performs slightly worse than non-exploring MCTS in case of low numbers of actions per turn, it outperforms all other methods with increasing numbers of actions. Additionally, the user study suggests that OEP also performs well against human players. Only OEP was tested against humans.

The paper begins with a brief review of relevant related work. Section III describes the testbed used in our experiments, which is a game called *Hero Academy*. Section IV describes OEP, which is followed by a number of different tree search approaches including MCTS. Next, Section VII presents the experimental setup and results and finally the paper concludes with a discussion (Section VIII) and conclusion (Section IX).

II. Related Work and Background

This section reviews relevant work on Monte Carlo Tree Search and evolutionary algorithms that runs while the agent is playing a game. There does not exist much work on multi-action adversarial games in the literature.

A. Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a best-first search that uses stochastic sampling as a heuristic [13], [16], and has been successfully applied to games with large branching factors such as Civilization II [17], Magic the Gathering [18] and Settlers of Catan [19]. The algorithm starts with a root node representing the current game state. Four phases are sequentially executed in iterations until a given time budget is used or a satisfying goal state is reached. In the selection phase, the tree is traversed from the root node using a *tree policy* until a node with unexpanded children is reached. In the expansion phase, a child node is expanded from the selected node. In the simulation phase (also called rollout), the remaining part of the game, from the expanded node's game state, is played out using a *default policy*. In the backpropagation phase, the outcome of the game is backpropageted up the tree until the root node is reached.

The tree policy determines how the search balances exploration and exploitation during the selection phase. Usually the Upper Confidence Bounds (UCB) algorithm is used [20], which selects the node that maximizes:

$$\text{UCB1} = \overline{X}_j + C\sqrt{\frac{2\ln n}{n_j}},$$

where n is the visit count of the current node, n_j is the visit count of the child j, C is a constant determining the amount of exploration versus exploitation and $\overline{X}_j$ is the normalized value of child j. The default policy is used during rollouts to select actions, which can be a complex scripted policy or one that selects random actions. An ϵ-greedy strategy can also be used to select a random action at probability ϵ and at probability $1-\epsilon$ follows some predefined policy.

MCTS has shown promise for many non-adversarial games as well, in particular with high branching factors, hidden information, and/or non-deterministic outcomes. To allow MCTS to be applied to games with increasingly higher branching factors, a variety of different MCTS variations has been developed. Numerous enhancements exist for MCTS to handle large branching factors, such as First-Play Urgency (FPU) [21], which encourages exploitation in the early stages by assigning a fixed score to unvisited nodes. Another enhancement that has been shown to improve MCTS in Go is Rapid Action Value Estimation (RAVE) [22] which updates statistics in nodes, with a decreasing effect, when their corresponding action is selected during rollouts. Portfolio Greedy Search [23] and Hierarchical Portfolio Search [24] introduced a Script-based approach, which have also been applied to MCTS [25], which deals with large branching factors in real-time strategy games by exploring a search space of scripted behaviors instead of actions. NaïveMCTS builds a tree where each node corresponds to a combination of actions, and the exploration policy is based on a naive assumption that a unit's actions are independent of other units' actions [4]. Portfolio Greedy Search and NaïveMCTS require that actions must be tied to units, as is common in real-time strategy games. Progressive strategies have been used to limit the search space with success in Go [26] by focusing the search using domain knowledge and then slowly unpruning nodes. Several progressive pruning methods have shown to improve MCTS for Go [27], where the idea is to prune nodes that are statistically inferior to their siblings. Sequential halving splits the time budget into a number of phases wherein exploration happens in a uniform manner and after each phase the worst half of the nodes are eliminated [28]. MCTS can use macro-actions (repeated actions) to reduce the depth of the search tree, which can be beneficial in domains that require continuous control [29].

B. Rolling Horizon and Online Evolutionary Algorithms

Evolutionary algorithms have been used to evolve controllers for numerous games [8]. Usually learning happens *offline* as a fixed behavior is evolved in a training phase, while evolution is not applied during the game. Genetic programming has been used to evolve programs that can perform planning, where each candidate planner is evaluated by simulating the outcome of its generated plan [30]. Perez et

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

al. introduced an evolutionary algorithm called *rolling horizon evolution* that runs *online* while the agent is playing to evolve action sequences. Rolling horizon evolution has been applied, with good results, to a number of real-time environments including the Physical Traveling Salesman Problem [10] and many games in the General Video Game Playing benchmark [11], [12]. Rolling horizon evolution evolves a sequence of actions for a fixed number of steps into the future. After the time budget is used, the first action in the most fit action sequence is performed, where after new actions sequences are evolved from scratch one step further into the future, i.e. the horizon is "rolling". The evolved action sequences are evaluated by simulating these in a forward model and evaluating the outcome. This paper introduces our algorithm Online Evolutionary Planning (OEP), which can be applied to multi-action adversarial games, and was first introduced in a conference paper in Justesen et al. [15]. OEP is more general than rolling horizon evolution as it does not include the "rolling" approach. Instead, the entire time budget is used to evolve an action sequence for a complete turn, which will be performed to end. This paper will show that this algorithm works well in the turn-based adversarial game Hero Academy. Building on our previous conference paper [15], a portfolio-based version of OEP has already been tested for small-scale battles in *StarCraft* [31] and a continual variation of OEP for build order planning [32].

Another evolutionary algorithm that runs online while the game is being played is *real-time Neuroevolution of Augmenting Topologies* (rtNEAT) [33]. This approach improves the behavior of multiple agents as the game is being played by replacing one individual every few game ticks with an offspring of the two most fit individuals. This method is however not directly applicable to the problem of searching for action sequences and is thus not a planning algorithm.

III. Testbed Game: Hero Academy

Before we describe our evolutionary algorithm and the two new MCTS variations in more detail, the testbed used for the experiments in this paper is described, which is a game called *Hero Academy*[1]. In order to run experiments efficiently, we use a simple clone of Hero Academy called *Hero AIcademy*[2].

The two-player turn-based tactics game Hero Academy is inspired by chess, with battles similar to the ones in the *Heroes of Might & Magic* series. An example game state in the Hero AIcademy implementation is shown in Figure 1. Each player has a pool of combat units and spells at their disposal, which they can deploy and use on a grid-shaped battle field of 9×5 squares. Special squares on the battlefield can boost a unit's attributes, while others allow the deployment of more units. Different classes of units have different combat roles, which allows players to employ a variety of different tactics. For example, the Council team has fighters, which are robust close-combat units that knock opponents back and wizards that can cast a powerful chain lightning spell, striking multiple units at once. Other units include archers, which are long-ranged units,

[1]http://www.robotentertainment.com/games/heroacademy/
[2]https://github.com/njustesen/hero-aicademy



Fig. 1: The user interface of Hero AIcademy showing a battlefield of 9×5 squares with two deploy zones (yellow arrows) on each side, two crystals and a number of units on each team. The symbols in the bottom represent the player's hand, and the numbers below the doors show the deck sizes.

clerics whose spells can heal friendly units and a single ninja, a powerful close-combat unit with the ability to swap position with a friendly unit through teleportation. Each player has a hand of up to six cards and a deck from which they draw new cards each turn. Each card symbolizes either a unit, an item or a spell.

The most central mechanic in the game is the usage of *action points* (AP). Each turn, the active player starts with five AP, which can be freely distributed among a number of different types of actions. These types are:

1) **Deployment**: A unit can be deployed from the hand of cards onto an unoccupied deploy zone.
2) **Movement**: One unit can be moved a number of squares equal to or lower than its *speed* attribute.
3) **Attacking**: One unit can attack an opponent unit within the number of squares equal to its *attack range* attribute.
4) **Spell casting**: Each team has one unique spell that can be cast from the hand onto a square on the board, where after the spell card is discarded.
5) **Swapping a card**: A card on the hand can be shuffled into the deck in hopes of drawing other cards in the following round.
6) **Special**: Some units have special actions such as healing and teleportation. We will refer to *healing* as a unique action type in this paper.

Especially noteworthy is that a player may choose to distribute more than one action point per unit, i.e. let a unit act twice or more times per turn. Because players make multiple actions per turn, we call it a *multi-action game*. The first player to eliminate all enemy units or *crystals* wins the game. When a unit looses all of its health it is not immediately removed from the game, but it instead becomes knocked down. Knocked down units have to be healed within one turn otherwise they are removed from the game. Units from the other team can however spend an AP during their turn to move a unit onto the square of a knocked down unit to remove it immediately, which is called *stomping*.

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

Because there are two win conditions, players can either go for one of them or try balance their strategy throughout the game. However, the key challenges in the game are the puzzles of finding the optimal action combination each turn. A clever action combination using several units and different types of actions can result in critical turnarounds during the game, and it is usually hard to plan several turns ahead.

A. Complexity Analysis

Due to the action point mechanic in Hero Academy, which makes the number of future game states significantly higher than in other games, the game is challenging for decision-making algorithms. Different combinations of actions can however, result in the same game state. It is hardly feasible to determine the exact branching factor as the number of allowed actions for a unit highly depends on the configuration of units on the board. Instead, it is trivial to estimate the branching factor by counting the number of possible actions in a recorded game. By doing this we have estimated the branching factor to be 60 on average. The average branching factor per turn can thus be estimated to be $60^5 = 7.78 \times 10^8$ as players have five actions per turn. Also based on observation we estimated the game length to be around 40 rounds on average. The game-tree complexity can thus be estimated to $((60^5)^2)^{40} = 1.82 \times 10^{711}$ (The branching factor of a turn is squared since both players take turn during a round). The game-tree complexity of Chess is "just" 10^{120} [34]. Another interesting complexity measure is the size of the state space, i.e. the number of possible board configurations. To simplify this estimation items and cards are ignored and only situations with all four crystals on the board with full health are considered. There are $45 - 4 - 2 = 39$ possible squares to place a unit on a 9x5 board with two crystals and deploy zones for each player. There can be between 0 and 26 units on the board, each with a health value between 0 and around 800, which gives us the following formula for estimating the state-space complexity:

$$\prod_{n=0}^{26} \prod_{i=1}^{n} ((39 - i + 1) \times 800) = 1.57 \times 10^{199}. \quad (1)$$

Since the board configuration is only one part of the game state and items and cards are not considered, the state-space complexity of Hero Academy is much larger than our estimated lower bound. As a comparison, Chess has a state-space complexity of 10^{43} [34].

Hero Academy also introduces hidden information, as the opponent's cards are unknown, as well as the order of cards in the deck. Stochastic games with hidden information can be approached with various forms of determinization methods [35], [36]. In this paper, we ignore the aspect of hidden information and randomness since we are only interested in ways to deal with the complexities of multi-action games.

IV. Online Evolutionary Planning

In this section we present an evolutionary algorithm that, inspired by the rolling horizon evolution, evolves strategies while it plays the game. We call this algorithm *Online Evolutionary Planning* (OEP), and we have implemented it to play Hero Academy, where it aims to evolve optimal action sequences every turn. Each genome in a population thus represents a sequence of five actions. An exhaustive search is not able to explore the entire space of action sequences within a reasonable time frame and may miss many interesting choices. An evolutionary algorithm on the other hand can explore the search space in a very different way.

Algorithm 1 Online Evolutionary Planning (EOP) for multi-action adversarial games (Procedures PROCREATE CROSSOVER, MUTATION, CLONE and EVAL are omitted for brevity)

```
1:  procedure ONLINEEVOLUTIONARYPLANNING(State s)
2:      Genome[] pop = ∅                                  ▷ Population
3:      INIT(pop, s)
4:      while time left do
5:          for each Genome g in pop do
6:              State clone = CLONE(s)
7:              clone.update(g.actions)
8:              if g.visits = 0 then      ▷ EVAL is deterministic
9:                  g.value = EVAL(clone)
10:             g.visits++
11:         sort pop in descending order by value
12:         pop = first half of pop                       ▷ 50% Elitism
13:         pop = PROCREATE(pop)          ▷ Mutation & Crossover
14:     return pop[0].actions                  ▷ Best action sequence
15:
16: procedure INIT(Genome[] pop, State s)
17:     for x = 1 to POP_SIZE do
18:         State clone = CLONE(s)
19:         Genome g = new Genome()
20:         g.actions = RANDOMACTIONS(clone)
21:         g.visits = 0
22:         pop.add(g)
23:
24: procedure RANDOMACTIONS(State s)
25:     Action[] actions = ∅
26:     Boolean p1 = s.p1                          ▷ Who's turn is it?
27:     while s is not terminal AND s.p1 = p1 do
28:         Action a = random available action in s
29:         s.update(a)
30:         actions.push(a)
31:     return actions
```

An overview of our online evolutionary planning algorithm is presented next, which is also shown in pseudocode (Algorithm 1). Evolutionary algorithms iteratively optimize an initially randomized population of candidate solutions (genomes) through recombination and selection based on elitism. When applied to planning in Hero Academy games, the genotype of each genome in the population is a vector of five actions, where one action represents a type and one or more locations if needed. An example of a genotype is: [**Move**$((0,4) \rightarrow (2,4))$, **Heal**$((2,4) \rightarrow (4,4))$, **Heal**$((2,4) \rightarrow (4,4))$, **Attack**$((4,0) \rightarrow (6,1))$, **Deploy**$(0 \rightarrow (0,4))$] which is also visualized in Figure 2. Note, that identical attack or heal actions can be repeated to deal more damage or gain more health. Locations given in

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

	Archer	Cleric	Knight	Ninja	Wizard
Equipment:					
Dragonscale	30	30	30	30	20
Runemetal	40	20	-50	20	40
Helmet	20	20	20	10	20
Scroll	50	30	-40	40	50
Special squares:					
Assault	40	10	120	50	40
Deploy	-75	-75	-75	-75	-75
Defense	80	20	30	60	70
Power	120	40	30	70	100

TABLE I: Bonuses added when units are equipped (used by $eq(u)$), and bonuses added when units stand on special squares (used by $sq(u)$).

two dimensions are on the board (from the top left square) and in one dimension are cards on the hand (from left to right). The phenotype is the resulting game state after taking these actions in the current game state.

The initial population is composed of random genomes, which are created by repeatedly selecting random actions based on the given forward model. This process is repeated until no more action points are left. After the creation of the initial population, the population is improved over a large number of generations until a given time budget is exhausted.

In this paper, Hero AIcademy itself serves as the forward model and the fitness of an action sequence is calculated as the difference between the values of both players' units. Both the units on the game board as well as those still at the players' disposal are taken into account. The assumption behind this particular fitness function is that the difference in units serves as a good indicator for which player is more likely to win. In more detail, the value $v(u)$ of unit u is calculated as follows:

$$v(u) = u_{hp} + \underbrace{u_{maxhp} \times up(u)}_{\text{standing bonus}} + \overbrace{eq(u) \times up(u)}^{\text{equipment bonus}} + \underbrace{sq(u) \times (up(u) - 1)}_{\text{square bonus}}, \tag{2}$$

where u_{hp} is the unit's number of health points, $sq(u)$ is a bonus based on the type of the square the unit stands on, and $eq(u)$ is a bonus that depends on the unit's equipment. The exact modifiers are shown in Table I. The modifying term $up(u)$ gives a negative reward for knocked down units:

$$up(u) = \begin{cases} 0, & \text{if } u_{hp} = 0 \\ 2, & \text{otherwise} \end{cases}. \tag{3}$$

After the evaluation, the genomes with the lowest scores are removed from the population. Each one of the remaining genomes is paired with another randomly selected genome, creating a new offspring through uniform crossover. Figure 3 shows the crossover between two example action sequences in Hero Academy. Because a naive crossover can lead to illegal action sequences, the crossover checks for the legality of a move when two sequences are combined. For example, to be able to move a unit from a certain position on the board it is required that a unit in fact stands on that particular square. However, this precondition might not be fulfilled due to an earlier action in the sequence. To avoid such situations, actions are only selected from a randomly chosen parent if it can be performed legally, otherwise the action will be taken from the other parent. If neither one of the two actions results in a legal move, the next action in the parent's sequence is chosen. In case this fails as well, a completely random available action is selected. Additionally, a certain proportion of the created offspring is mutated to introduce new actions to the population. Mutation changes one randomly chosen action to another legal action. If this results in an illegal action sequence, the following part of the sequence is changed to random but legal actions. Legality is checked by requesting the framework for available actions at each state by traversing the offspring's actions sequence.

To incorporate information about possible counter moves, attempts were made to base the heuristic on rollouts. Here fitness is determined by performing one rollout with a depth limit of five actions, corresponding to one turn. When a genome is tested more than once (because it has survived several generations in the population), the lowest value found in the genome's evaluations is used. The rating of an action sequence thus depends on the best of the known counter-moves. Because our experiments did not show any significant difference between a stochastic rollout as a fitness measure and a static evaluation, the later was chosen for the experiments in this paper. The large branching factor in this game is possibly the reason why evaluations are unreliable when based on a low number of rollouts.

V. Tree Search

A game tree can be described as an acyclic directed graph with the root node being the current game state. Edges in the graph represent available actions in the game that lead from one state to other hypothetical future game states. Therefore, the number of edges from a node corresponds to the number of actions available to the *active player* in that game state. Nodes also have certain values assigned to them, where higher values indicate more desirable game situations. For many adversarial games, in which the utility of one agent is the opposite of the other, agents take turns and thus the active player alternates between plies of the tree. In these situations, the well-known Minimax algorithm can be applied. However, in Hero Academy players can take several actions before the end of their turn. A potential tree search setup for Hero Academy could be to encode multiple actions as a joint action (i.e. an array of actions) that is assigned to each edge. However, because of the high number of possible permutations and therefore increased branching factor, we decided to model each action as its own node, essentially trading tree breath for depth.

In the following, we will present five game-playing tree-search methods for Hero Academy. Two of these are simple game-tree based methods, which were used as baselines, followed by MCTS including two novel variations.

Greedy Action. The *Greedy search among actions* method is the simplest of the developed methods. Greedy Action performs a one-ply search among all possible actions, and based on the employed heuristic (Equation 2) selects the action

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games



(a) Initial state (b) Evolved actions (c) Resulting state

Fig. 2: An example of an evolved action sequence that demonstrates that Online Evolutionary Planning can find solid action sequences in Hero Academy. In this example a critical combination is found that first heals the knocked down knight at (4,4). Because the knight afterwards stands on an assault square, any damage towards crystals are doubled which the archer utilizes by a lethal strike. (a) shows the initial state in the beginning of the red player's turn and (b) shows the actions evolved by Online Evolutionary Planning. The exact action sequence, shown in (b), is: [**Move**$((0,4) \rightarrow (2,4))$, **Heal**$((2,4) \rightarrow (4,4))$, **Heal**$((2,4) \rightarrow (4,4))$, **Attack**$((4,0) \rightarrow (6,1))$, **Deploy**$(0 \rightarrow (0,4))$]. The resulting state after performing the evolved action sequence and ending the turn is shown in (c).

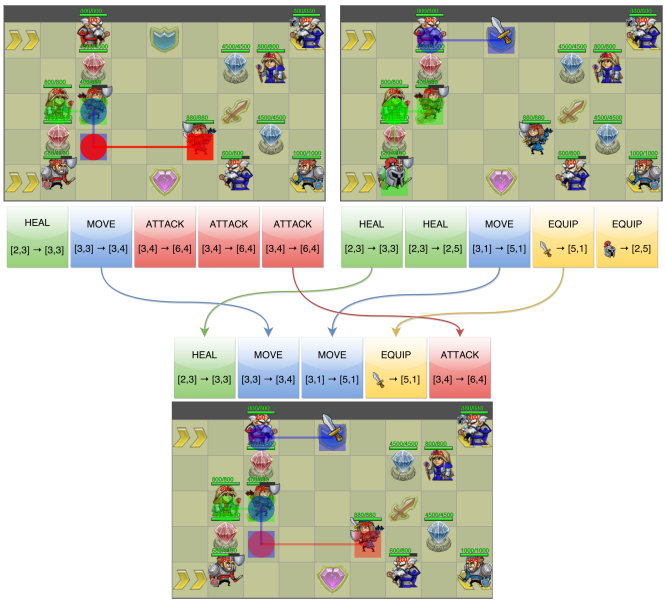


Fig. 3: Uniform crossover example in Hero Academy. Genes (actions) are randomly picked from the two parents.

that leads to the most promising game state. The search is invoked five times to complete a turn.

Greedy Turn. The *Greedy search among turns* performs a five-ply depth-first search, which corresponds to a full turn. The same heuristic as for Greedy Action rates all leaf nodes states and selects the action sequence that leads to the highest-rated state. A transposition table keeps track of already visited game states so that they are not visited again. Because this search is usually not exhaustive, choosing which actions to try first (i.e. action sorting) is critical.

Vanilla MCTS. Following the two greedy search variants, the vanilla MCTS method was implemented with an action based approach. In other words, one ply in the tree represents an action, not a turn. Therefore, a search with a depth of five has to be performed to reach the beginning of the opponent's turn. Our vanilla MCTS agent implements the four phases described in Section II-A, which follows the traditional MCTS algorithm using UCB [14]. The standard MCTS backpropagation was modified to handle two players with multiple actions. Our approach [15] is an extension of the *BackupNegamax* [14] algorithm (see Algorithm 2). It uses a list of edges reflecting the traversal during the selection phase, a Δ value corresponding to the result of the simulation phase and a boolean $p1$ that is *true* if player one is the max player and *false* otherwise.

Algorithm 2 A multi-action game modification of the BackupNegamax algorithm [15].

```
1:  procedure MultiNegamax(Edge[] T, Double Δ, Boolean p1)
2:      for all Edge e in T do
3:          e.visits++
4:          if e.to ≠ null then
5:              e.to.visits ++
6:          if e.from = root then
7:              e.from.visits ++
8:          if e.p1 = p1 then
9:              e.value += Δ
10:         else
11:             e.value −= Δ
```

An ϵ-greedy approach is employed in the rollouts that combines random play with the highest rated action based on the amount of damage dealt/healed. Rollouts that stop prior to a terminal state are evaluated by our heuristic (Equation 2).

As players in Hero Academy have to select five actions, two different approaches were tried: in the first approach the agent is invoked five times sequentially, with each iteration using a fifth of the total time budget, where after the actions are executed in the order found. However, another approach was found to be superior that uses the entire time budget, where after the tree is traversed from the root by selecting the best five nodes (actions) until the opponents turn is reached. This will allow the tree to expand as much as possible before actions are selected, and thus more options that are complete will be considered within the time budget.

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

The following describes two novel exploration-constrained variations of MCTS. We will refer to MCTS without these extensions as Vanilla MCTS.

Non-exploring MCTS. The search trees created by MCTS will barely be able to reach into the opponent's turn due to the complexity of the game. To overcome this we have developed a variation of MCTS that uses a non-exploring tree policy, i.e. $C = 0$, in combination with deterministic rollouts. All children of a node are still visited at least once before any of them are expanded further. This ensures that a very limited form of exploration still occurs, and since rollouts are deterministic, controlled by a greedy policy, it is acceptable to value children based only on one visit. If stochastic rollouts were used instead, some branches would never be revisited when rollouts happen to return unlucky outcomes.

Bridge Burning MCTS. Another novel approach to MCTS in multi-action games is what we call the Bridge Burning MCTS (BB-MCTS). This approach splits the time budget into a number of sequential phases equal to the number of actions in a turn. During each phase, the search functions as an ordinary MCTS search. However, in the end of each phase, all but the most promising node from the root are pruned and will never be added again. Another way to implement the same behavior is to treat the most promising child node as the root of the tree in the following phase. This approach is thus an aggressive progressive pruning strategy that will enable the search to reach deeper plies with the drawback of ignoring parts of the search space. The name *Bridge Burning* emphasizes that the nodes are aggressively pruned and can never be visited again. Figure 4 shows how nodes are pruned in three phases in a multi-action game with 3 actions in a turn.

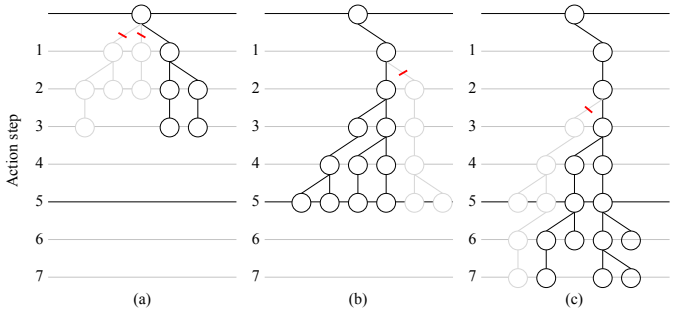


Fig. 4: An example of how nodes are progressively pruned using the bridge burning strategy in three phases to force the search to reach deeper plies (action steps). (a) After the first phase, all nodes except the most promising in ply one are pruned. (b,c) The search continues where after nodes are pruned one ply deeper.

For the tree-search methods, it makes sense to investigate the most promising moves first and thus a method for sorting actions is useful. A simple way would be to evaluate the resulting game state of each action, but this is usually a slow method. Instead, attack and spell actions are rated by how much damage they will deal, heal actions (including healing potions) by how many health points they will heal, equip actions by $p_u \frac{h_u}{m_u}$, where p_u, h_u and m_u is the power (equivalent to damage output) of the equipped unit u, health points and maximum health points. Movement actions are given a 30 point rating if movement is to a special square type and 0 otherwise. If an enemy unit is removed from the game, it is given a $2m_u$ point rating. If a knocked down unit u is healed, the rating of the action is $m_u + |e_u| \times 200$ where $|e_u|$ is the number of items carried by the healed unit u.

BB-MCTS is to some extend similar to an extreme implementation of sequential halving where only one node survives, and the number of phases is fixed to the number of actions.

VI. Experimental Setup

Here the experimental setup is described, which is the basis for the results presented in the next section. Each method played 100 games (as the Council team) against all other methods, 50 games as the starting player and 50 games as the second player. Figure 1 shows the map used. In contrast to the original game, Hero AIcademy was configured to be deterministic and without hidden information to focus our experiments on the challenge of performing multiple actions. Each method was limited to one processor and had a time budget of six seconds each turn. The winning percentages of each matchup counted draws as half a win for each player. While the rules of the original Hero Academy do not include draws, in these experiments a draw was called if no winner was found in 100 rounds. The experiments were carried out on an Intel Core i7-3517U CPU with 4 $\times$ 1.90GHz cores and 8 GB of ram. The specific configurations for the implemented game-playing methods were as follows:

Vanilla MCTS. The traditional UCT tree policy $\overline{X}_j + 2C\sqrt{\frac{2\ln n}{n_j}}$ was employed with an exploration constant of $C = \frac{1}{\sqrt{2}}$. The default policy was ϵ-greedy, where ϵ=0.5. A transposition table was implemented with the *descent-path only* backpropagation strategy. Thus, values and visit counts are stored in edges [37]. In fact, n_j in the tree policy is extracted from the child edges instead of the nodes. Rollouts were depth-limited to one turn, following the heuristic state evaluator described in Section IV. Preliminary experiments showed that short rollouts are preferred over long rollouts for MCTS in Hero Academy and that rollouts of just one turn show the best performance. Additionally, by adding some domain knowledge to the default policy through a specific ϵ-greedy strategy, the performance improves; ϵ-greedy selects a greedy action equivalent to the highest rated action by the action sorting method with a probability of ϵ, and a random action otherwise.

BB-MCTS. Same as for Vanilla MCTS, but with the Bridge Burning strategy as well.

Non-exploring MCTS. Same as for Vanilla MCTS, but with the exploration constant $C = 0$ and the default policy ϵ-greedy, where ϵ=1, such that rollouts are deterministic following the action sorting heuristic (Section V).

Online Evolutionary Planning (OEP). The population size was 100 with a survival rate of 0.5, a mutation probability of 0.1 and a uniform crossover operator. These parameters were found through prior experimentation.

The MCTS-based methods and OEP employ *action pruning*, which reduces the large search space of a turn by removing

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

	Random	Greedy Action	Greedy Turn	Vanilla MCTS	Non-expl. MCTS	BB-MCTS	OEP
Greedy Action	100%	-	36%	51.5%	2%	7.0%	2.0%
Greedy Turn	100%	64.0%	-	88.0%	23.0%	26.5%	19.5%
Vanilla MCTS	100%	48.5%	22.0%	-	0.0%	4.5%	2%
Non-exploring MCTS	100%	98.0%	77.0%	100%	-	80.0%	58.0%
BB-MCTS	100%	93.0%	73.5%	95.5%	20.0%	-	48.0%
OEP	100%	90.0%	80.5%	98%	42.0%	52.0%	-

TABLE II: Win percentages of the agents listed in the left-most column in 100 games against agents listed in the top row. A win percentage of 62% or more is significant with a significance level of 0.05 using the Wilcoxon Signed-Rank Test.

(pruning) redundant swap actions and sub-optimal spell actions from the set of available actions in a state. Swap actions are redundant if they swap the same kind of item (i.e. they will produce the same outcome) and can thus be removed. Additionally, spell actions are sub-optimal if other spell actions cover the same or more enemy units.

VII. Results

Our results, shown in Table II, show the performance of each method. The non-exploring MCTS is the best performing method but with no significant difference compared to Online Evolutionary Planning (OEP) and BB-MCTS, which have a similar performance. Vanilla MCTS plays on the same level as the Greedy Action baseline, which indicates that it is able to identify the action that gives the best immediate reward while it is unable to search sufficiently through the space of possible action sequences. All methods convincingly beat random search. A video of OEP playing against the Greedy Action baseline has been uploaded to YouTube[3].

A. Search Characteristic Comparison

To get further insights into how the different methods explore the search space, the number of different action sequences each method is able to evaluate within the given time budget was tracked. Since many action sequences produce the same outcome, only the number of unique outcomes evaluated by each method were counted and only those after taking five actions. The *Greedy Turn* search evaluated 579,912 unique outcomes on average during a turn. OEP evaluated on average 9,344 unique outcomes and MCTS only 201. Each node at the fifth ply of the MCTS tree corresponds to one outcome and the search only manages to expand the tree to a limited number of nodes at this depth. Further analysis reveals that the average depth of leaf nodes in the final MCTS trees is 4.86 plies, while the deepest leaf node of each tree reached an average depth of 6.38 plies. This means that the search tree just barely enters the opponents' turn even though it manages to run an average of 258,488 iterations per turn. OEP ran an average of 3,693 generations each turn but appeared to get stuck at local optima quickly due to the low number of unique outcomes evaluated. These results suggests that OEP would play almost equally good with a much lower time budget, but also that there could be room to improve the algorithm itself.

B. Changing the Number of Actions

Multi-action games can have many forms and Hero Academy is just one example. An additional experiment was performed in which our methods were tested in variations of Hero Academy. The rules were altered by changing the number of action points (AP) per turn to 1, 3, 5, 10, 15, 20 and 25. This also increases the complexity of one turn exponentially. The time budget in this experiment was set to 2000 ms. and the turn limit to $\frac{5}{AP} \times 100$. Reaching the turn limit results in a draw, which is counted as half a win for both players. Only MCTS with our two variations as well as OEP are included in this experiment as it makes the most interesting comparison. The results, which are plotted on Figure 5, show the win percentage of OEP against each MCTS variation in 100 games. The results show that OEP handles the increased number of action points best with a win rate of 55% or more with 10 or more AP against any of the other methods. This indicates that OEP has the best scalability in terms of complexity in multi-action games. Increasing the number of AP to 20 and more makes it possible to win the game in a few turns. This make the outcome of the game highly depend on who gets to start. Vanilla MCTS does however not show that it is able to identify these fast win strategies.

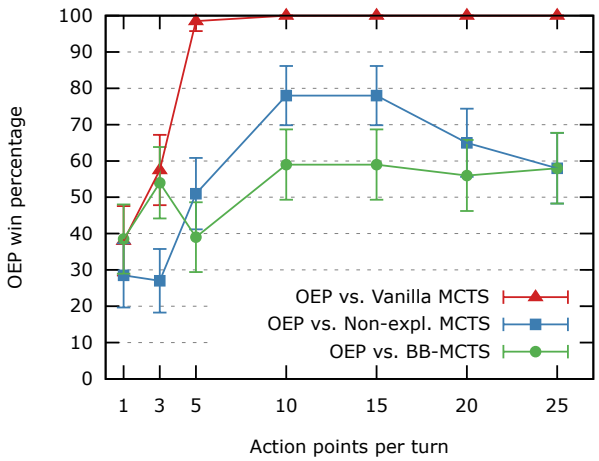


Fig. 5: The average win percentages of Online Evolutionary Planning (OEP) versus BB-MCTS, Non-exploring MCTS and Vanilla MCTS. 100 games were played in each matchup for each game configuration, which each had a different number of AP per turn. This shows that MCTS, including the two variations, perform best with few AP per turn while OEP performs best with many AP per turn. Error bars show 95% confidence intervals.

C. Versus Human Players

One important and often overseen experiment is to test game-playing algorithms against human players. Such com-

[3]https://www.youtube.com/watch?v=WYGwBbccYCQ

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

Skill level	Human wins (a)	Human wins (b)
Beginner	13/26 (50%)	19/45 (42.2%)
Intermediate	12/13 (92%)	15/19 (78.9%)
Expert	16/16 (100%)	20/21 (95.2%)
All	41/55 (80%)	**54/85 (63.5%)**

TABLE III: The number of wins by human players of various skill levels in 85 games against Online Evolutionary Planning. (a) Shows results from games that ended with a winner, and (b) also includes games where the player quit prematurely.

parisons can provide important insights about the current state of a method, and whether there is promise implementing it in a real game product. Since OEP, BB-MCTS and non-exploring MCTS show similar performance in games with 5 AP per turn, we suspect this is also the case when compared against humans. Thus in this paper we focus on only testing OEP against human players but believe a more comprehensive comparison that includes MCTS remains important future work. 111 games were recorded, which were played with hidden information and randomness as in the actual game. It makes no difference for OEP whether or not the game contains randomness as it only considers its own turn. The Hero AIcademy client was distributed on social networks and community web sites for Hero Academy players to reach a sufficient number of players with various skill level. The client was extended to push game events to a web server every turn to be stored in a database. Unfortunately, we do not know the number of participants, only the number of games played in total. The client asked players about their skill level prior to each game with the options *beginner*, *intermediate* and *expert*. If *beginner* was selected a very short introduction to the game was presented before the game started. 111 game records were collected and the results are shown in Table III. In 56 of the 111 games, the game was quit before a terminal state was reached, possibly as a way of surrendering. Of the games that reached the end, the human players won 41 out of 55 games. Games that were quit before a winner was found were evaluated using the heuristic described in Section IV to determine a winner. Games were however excluded if the heuristic estimated an advantage lower than 10% of the maximum to either of the players. By including these games, the human players won only 63.5% of the games showing that many players left the game while they were behind. These results show that OEP is competitive against human players while being inferior to expert players and some intermediate players.

VIII. Discussion

The results show that OEP performs better than all tree search methods for most settings of our benchmark, and as the number of actions per turn (and thus the branching factor) increases, the relative advantage of OEP over tree search seems to increase. The crucial question is why this is so. Answering this question will require much further research, and is likely to contribute to our understanding of planning in general with broad domain-general implications. Our current understanding is that tree search algorithms, which by definition start from the root node of the tree and explore outward, concentrate their search on the part of plan space closest to the root in problems with high branching factor. However, all parts of the plan are important. By seeing the plan as a string and searching the space of strings, the evolutionary planning approach ensures that the search considers full plans instead of focusing on the region around the origin.

Vanilla MCTS is unable to deal with the complexity of Hero Academy as the search space is simply too large. A similar conclusion has been made for the game Arimaa, in which player make four actions each turn [38]. We show that by constraining the exploration of MCTS, performance can be significantly improved. In the future, it will be interesting to compare our approaches to MCTS with optimized parameters as well as existing MCTS enhancements that have shown to work well in Go, such as progressive strategies [26], [27]. They would most likely need to be configured to constrain the exploration aggressively as our methods do. MCTS was not designed to deal with multiple actions each turn and little research has been done on problems of this type. Clearly more research is needed since many strategy games are multi-action games, which may also model real-life decision making.

One problem with OEP is that it does not take the opponent's turn into account. This could perhaps be achieved with competitive coevolution [39] by having another population for the seconds player's turn and let the genomes in each population compete when determining their fitness. One problem with this idea is that evolved actions sequences in the other population highly depends on the outcome of the action sequences in the first population (e.g. attacking a certain square is only effective if the opponent actually moves a unit there). Co-evolution has already been tested for the Rolling Horizon Evolution for games with small branching factors [40] and applying this to OEP for multi-action games is an interesting direction for future research. OEP reached an optimum with only a small amount of generations, which means that it either finds the global optimum or is stuck locally. Diversity maintenance methods (such as niching [41]) or Tabu search could perhaps improve the performance [42]. Finally, it would be very interesting to see how OEP performs in more complex multi-action games.

IX. Conclusion

This paper described and compared three methods for playing adversarial games with very large branching factors. Such branching factors, while extreme in comparison to classical board games, are common in strategy games and presumably also in the real-world scenarios they model. To tackle this challenge, we propose *Online Evolutionary Planning* (OEP). The core idea is to use an evolutionary algorithm to search for the next turn, where the turn is composed of a sequence of actions. We compared this algorithm with several other algorithms on the game Hero Academy; the comparison set includes Monte Carlo Tree Search, which is the state of the art for many games with high branching factor, as well as two new variations of MCTS designed to better handle the high branching factor. Non-exploring MCTS does not explore other paths than the currently most promising one and uses deterministic rollouts, while BB-MCTS deepens the search by periodically removing upper parts of the tree so that only

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCIAIG.2017.2738156, IEEE Transactions on Computational Intelligence and AI in Games

lower parts are explored. Our results show that both OEP and the two new MCTS variations convincingly outperform standard MCTS as well as a depth first search on this problem. As for the relative performance of the new methods, non-exploring MCTS slightly outperforms OEP on small numbers of actions, but as the number of actions per turn (and therefore the branching factor) is increased, OEP's superiority over other approaches increases. Wang et al. also conclude that OEP outperforms MCTS when the complexity of the problem increases [31]. We further tested OEP's ability to play against human players of various skill levels. OEP was able to win 31 games out of 85 (36.5%), and can play competitively against human players with a low to medium skill level while being easily outplayed by medium to high-level players. Future work will investigate how well this performance holds up in other games, and how to improve the evolutionary search.

REFERENCES

[1] J. v. Neumann, "Zur theorie der gesellschaftsspiele," *Mathematische Annalen*, vol. 100, no. 1, pp. 295–320, 1928.

[2] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[3] B. Bouzy and T. Cazenave, "Computer go: an AI oriented survey," *Artificial Intelligence*, vol. 132, no. 1, pp. 39–103, 2001.

[4] S. Ontanón, "The combinatorial multi-armed bandit problem and its application to real-time strategy games," in *Ninth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2013.

[5] D. Perez, S. Samothrakis, S. Lucas, and P. Rohlfshagen, "Rolling horizon evolution versus tree search for navigation in single-player real-time games," in *Proceedings of the 15th annual conference on Genetic and evolutionary computation*. ACM, 2013, pp. 351–358.

[6] J. Togelius, S. Karakovskiy, J. Koutník, and J. Schmidhuber, "Super Mario evolution," in *Computational Intelligence and Games, 2009. CIG 2009. IEEE Symposium on*. IEEE, 2009, pp. 156–161.

[7] L. Cardamone, D. Loiacono, and P. L. Lanzi, "Evolving competitive car controllers for racing games with neuroevolution," in *Proceedings of the 11th Annual conference on Genetic and evolutionary computation*. ACM, 2009, pp. 1179–1186.

[8] S. Risi and J. Togelius, "Neuroevolution in games: State of the art and open challenges," *IEEE Transactions on Computational Intelligence and AI in Games*, 2015.

[9] S. M. Lucas and G. Kendall, "Evolutionary computation and games," *Computational Intelligence Magazine, IEEE*, vol. 1, no. 1, pp. 10–18, 2006.

[10] D. Perez, P. Rohlfshagen, and S. M. Lucas, "Monte-carlo tree search for the physical travelling salesman problem," in *Applications of Evolutionary Computation*. Springer, 2012, pp. 255–264.

[11] R. D. Gaina, J. Liu, S. M. Lucas, and D. Pérez-Liébana, "Analysis of vanilla rolling horizon evolution parameters in general video game playing," in *European Conference on the Applications of Evolutionary Computation*. Springer, 2017, pp. 418–434.

[12] J. Levine, C. B. Congdon, M. Ebner, G. Kendall, S. M. Lucas, R. Miikkulainen, T. Schaul, T. Thompson, S. M. Lucas, M. Mateas *et al.*, "General video game playing." *Artificial and Computational Intelligence in Games*, vol. 6, pp. 77–83, 2013.

[13] G. Chaslot, S. Bakkes, I. Szita, and P. Spronck, "Monte-carlo tree search: A new framework for game AI." in *AIIDE*, 2008.

[14] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, S. Colton *et al.*, "A survey of monte carlo tree search methods," *Computational Intelligence and AI in Games, IEEE Transactions on*, vol. 4, no. 1, pp. 1–43, 2012.

[15] N. Justesen, T. Mahlmann, and J. Togelius, "Online evolution for multi-action adversarial games," in *Applications of Evolutionary Computation*. Springer, 2016, pp. 590–603.

[16] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *International Conference on Computers and Games*. Springer, 2006, pp. 72–83.

[17] S. Branavan, D. Silver, and R. Barzilay, "Non-linear monte-carlo search in Civilization II." AAAI Press/International Joint Conferences on Artificial Intelligence, 2011.

[18] P. I. Cowling, C. D. Ward, and E. J. Powley, "Ensemble determinization in monte carlo tree search for the imperfect information card game Magic: The gathering," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 4, pp. 241–257, 2012.

[19] I. Szita, G. Chaslot, and P. Spronck, "Monte-carlo tree search in Settlers of Catan," in *Advances in Computer Games*. Springer, 2009, pp. 21–32.

[20] P. Audouard, G. Chaslot, J.-B. Hoock, J. Perez, A. Rimmel, and O. Teytaud, "Grid coevolution for adaptive simulations: Application to the building of opening books in the game of go," in *Workshops on Applications of Evolutionary Computation*. Springer, 2009, pp. 323–332.

[21] S. Gelly and Y. Wang, "Exploration exploitation in go: UCT for monte-carlo go," in *NIPS: Neural Information Processing Systems Conference On-line trading of Exploration and Exploitation Workshop*, 2006.

[22] D. P. Helmbold and A. Parker-Wood, "All-moves-as-first heuristics in monte-carlo go." in *IC-AI*, 2009, pp. 605–610.

[23] D. Churchill and M. Buro, "Portfolio greedy search and simulation for large-scale combat in StarCraft," in *Computational Intelligence in Games (CIG), 2013 IEEE Conference on*. IEEE, 2013, pp. 1–8.

[24] ——, "Hierarchical portfolio search: Prismatas robust AI architecture for games with large search spaces," in *Proceedings of the Artificial Intelligence in Interactive Digital Entertainment Conference*, 2015.

[25] N. Justesen, B. Tillman, J. Togelius, and S. Risi, "Script-and cluster-based UCT for StarCraft," in *Computational Intelligence and Games (CIG), 2014 IEEE Conference on*. IEEE, 2014, pp. 1–8.

[26] G. M. J. Chaslot, M. H. Winands, H. J. van den Herik, J. W. Uiterwijk, and B. Bouzy, "Progressive strategies for monte-carlo tree search," *New Mathematics and Natural Computation*, vol. 4, no. 03, pp. 343–357, 2008.

[27] B. Bouzy, "Move-pruning techniques for monte-carlo go," in *Advances in Computer Games*. Springer, 2005, pp. 104–119.

[28] Z. Karnin, T. Koren, and O. Somekh, "Almost optimal exploration in multi-armed bandits," in *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, 2013, pp. 1238–1246.

[29] E. J. Powley, D. Whitehouse, and P. I. Cowling, "Monte carlo tree search with macro-actions and heuristic route planning for the physical travelling salesman problem," in *Computational Intelligence and Games (CIG), 2012 IEEE Conference on*. IEEE, 2012, pp. 234–241.

[30] J. R. Koza, "Genetically breeding populations of computer programs to solve problems in artificial intelligence," in *Tools for Artificial Intelligence, 1990., Proceedings of the 2nd International IEEE Conference on*. IEEE, 1990, pp. 819–827.

[31] C. Wang, P. Chen, Y. Li, C. Holmgård, and J. Togelius, "Portfolio online evolution in StarCraft," in *Twelfth Artificial Intelligence and Interactive Digital Entertainment Conference*, 2016.

[32] N. Justesen and S. Risi, "Continual online evolution for in-game build order adaptation in StarCraft," in *The Genetic and Evolutionary Computation Conference (GECCO)*, 2017.

[33] K. O. Stanley, B. D. Bryant, and R. Miikkulainen, "Real-time neuroevolution in the NERO video game," *IEEE transactions on evolutionary computation*, vol. 9, no. 6, pp. 653–668, 2005.

[34] C. E. Shannon, "XXII. programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 41, no. 314, pp. 256–275, 1950.

[35] T. Cazenave, "A phantom-go program," in *Advances in Computer Games*. Springer, 2005, pp. 120–125.

[36] P. Nijssen and M. H. Winands, "Monte carlo tree search for the hide-and-seek game Scotland Yard," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 4, pp. 282–294, 2012.

[37] A. Saffidine, T. Cazenave, and J. Méhat, "UCD: Upper confidence bound for rooted directed acyclic graphs," *Knowledge-Based Systems*, vol. 34, pp. 26–33, 2012.

[38] T. Kozelek, "Methods of MCTS and the game Arimaa," *Charles University, Prague, Faculty of Mathematics and Physics*, 2009.

[39] C. D. Rosin and R. K. Belew, "New methods for competitive coevolution," *Evolutionary Computation*, vol. 5, no. 1, pp. 1–29, 1997.

[40] J. Liu, D. Pérez-Liébana, and S. M. Lucas, "Rolling horizon coevolutionary planning for two-player video games," in *2016 8th Computer Science and Electronic Engineering (CEEC)*, 2016, pp. 174–179.

[41] S. W. Mahfoud, "Niching methods for genetic algorithms," *Urbana*, vol. 51, no. 95001, pp. 62–94, 1995.

[42] F. Glover and M. Laguna, *Tabu Search**. Springer, 2013.

1943-068X (c) 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.